Programski prevodioci

02 Leksička analiza

25-26

Dunja Vrbaški

Fakultet tehničkih nauka, Novi Sad

Leksička analiza - Skener

- Prva faza prevođenja: podeliti ulazni string na smislene delove

- Nazivi
- Skener – skeniramo tekst na ulazu, slovo po slovo, leksemu po leksemu
- Lekser – lekseme

```
if (x == 5)
{
   y = 3; //komentar
}
```

if	(	x	==	5	)	{	y	=	3	;	}

lekseme

Zašto su neke lekseme označene na slici?

```
if (x == 5)
{
   y = 3; //komentar
}
```

if		(	x		==		5	)		{		y		=		3	;				}

- Zapravo, prepoznajemo i beline
- U nekim jezicima – utiču na značenje
- Mogu biti od koristi za druge aktivnosti

Šta bi bila greška u leksičkoj analizi?

Šta bi bila greška u leksičkoj analizi?

```
if (8k == 5)
{
   y = 3 * 2;
}
```

Ako je naš jezik takav da:

- identifikator ne može da počne brojem niti numerička vrednost može da se završi slovom *k*
- ne prepoznaje * kao deo jezika

Šta znači “deo jezika”?

Tokeni - lekseme se klasifikuju po tipu uz eventualno dodavanje korisnih informacija

Leksema	Vrsta tokena	Vrednost tokena
if	Ključna reč	if
while	Ključna reč	while
=	Dodela	
==	Relacioni operator	==
<	Relacioni operator	<
+	Aritmetički operator	+
;	Tačka zarez	
x	Identifikator	x
y	Identifikator	y
5	Literal	5

Vrsta tokena se u često zove i klasa tokena

Token je par kog čine: vrsta tokena i vrednost tokena.

- Vrsta tokena je apstraktna reprezentacija klase jezičkih jedinica.
- Vrednost tokena nije obavezna i najčešće predstavlja konkretnu, prepoznatu, vrednost za konkretan ulazni string.

Lekseme su instance tokena koje su prepoznate prilikom obrade ulaznog stringa.

Leksema	Vrsta tokena	Vrednost tokena
if	Ključna reč	if
while	Ključna reč	while
=	Dodela	
==	Relacioni operator	==
<	Relacioni operator	<
+	Aritmetički operator	+
;	Tačka zarez	
x	Identifikator	x
y	Identifikator	y
5	Literal	5

Leksema	Vrsta tokena	Vrednost tokena
if	Ključna reč	if
while	Ključna reč	while
=	Operator	=
==	Operator	==
<	Operator	<
+	Operator	+
;	Tačka zarez	
x	Identifikator	x
y	Identifikator	y
5	Literal	5

Primer implementacije

Leksema	Vrsta tokena (neformalno)	Vrsta tokena (npr enum)	Vrednost tokena
if	Ključna reč	KW	If
while	Ključna reč	KW	while
=	Dodela	ASG	
==	Relacioni operator	ROP	==
<	Relacioni operator	ROP	<
+	Aritmetički operator	AOP	+
;	Tačka zarez	SC	
x	Identifikator	ID	x
y	Identifikator	ID	y
5	Literal	LIT	5

Primer implementacije

Leksema	Vrsta tokena (neformalno)	Vrsta tokena (npr enum)	Vrednost tokena
if	Ključna reč	KW	If
while	Ključna reč	KW	while
=	Dodela	ASG	
==	Relacioni operator	ROP	==
<	Relacioni operator	ROP	<
+	Aritmetički operator	AOP	+
;	Tačka zarez	SC	
x	Identifikator	ID	x
y	Identifikator	ID	y
5	Literal	LIT	5

Kog tipa su vrednosti?

- Token može evidentirati i poziciju u izvornom kodu radi informativnih poruka o greškama. Pamti se: linija // linija i kolona // pomeraj od početka teksta

- Mogu se pamtiti i druge informacije (atributi).

Zašto, uopšte, prepoznajemo i pamtimo tokene?

Razmisliti:
Za leksičku analizu nam zapravo tokeni nisu neophodni.

Dovoljno je da delove stringa prepoznamo kao elemente jezika i javimo da li ima grešaka ili ne tj da li je program napisan korišćenjem ispravnih reči.

→ Trebaju nam u nastavku

Kako implementirati skener?

Kako implementirati skener?

- čitamo string sa ulaza, karakter po karakter
- prepoznajemo lekseme
- (kreiramo tokene)
- informišemo o rezultatu (ok // greška)

Kako implementirati skener?

- čitamo string sa ulaza, karakter po karakter
- prepoznajemo lekseme
- (kreiramo tokene)
- informišemo o rezultatu (ok // greška)

- ručno (while, if/switch, lookahead) → proizvoljno
- alat (flex, lex, antlr...) → treba nam neka formalizacija jezika

Bez obzira na pristup, pojavljuju se određeni problemi. Na primer:

- potrebno je da <= bude ispravno prepoznato (ne kao 2 operatora već kao jedan)
- ako postoje rezervisane reči potrebno je da one ne budu prepoznate kao opšti identifikatori
- pojava greške – da li odmah stati sa obradom i prijaviti grešku ili dati informaciju i pokušati dalje sa analizom
- beline su važne ili ne
- signal za kraj (može biti važno za naredne faze)
- ...

Takođe, postavlja se pitanje šta je ulaz za sledeću fazu (sintaksna analiza)?

- obraditi ceo ulazni string i generisati niz tokena koji se u celosti predaje narednoj fazi
- prosleđivati, na zahtev, token po token narednoj fazi
- nešto treće

Mogu se koristiti regularni izrazi za opis leksike programskog jezika.
(formalizacija)

Zašto bismo to radili?

- Jednostavna formalna reprezentacija leksike jezika

- Flex - na osnovu leksike definisane regularnim izrazima automatski kreira skener.
- Ručno - možemo slično primeniti i iskoristiti za prepoznavanje leksema

U nastavku:
- regularni izrazi
- kako se realizuju skeneri na osnovu regularnih izraza
- kako se koristi Flex

Regularni izrazi se koriste za opis *regularnih jezika*.
Regularni jezici su vrsta formalnih jezika.

Još jedna vrsta formalnih jezika su, na primer, *kontekstno slobodni jezici*.

Jezik leksike programskih jezika predstavlja regularan jezik.
Velik deo sintakse programskih jezika predstavlja kontekstno slobodan jezik.

Teorija formalnih jezika
Teorija automata

Kasnije...

Oznaka	Primer	Formiranje	Skup reči
		Prazan string	
	a	Svaki simbol (karakter) je RE	
	ab	Vrši se konkatenacija dva RE	“ab”
*	ab*	RE na koji se odnosi se ponavlja 0 ili više puta	“a”, “ab”, “abb”,....
+	ab+	RE na koji se odnosi se ponavlja 1 ili više puta	“ab”, “abb”,...
?	ab?	RE na koji se odnosi se ponavlja 0 ili 1 put	“a”, “ab”

RE – Regular Expression, regularni izraz

Oznaka	Primer	Formiranje	Skup reči
|	a|b	Alternativa	“a”, “b”
( )	a(b|c)	Grupisanje	“ab”, “ac”
[ ]	[abc]	Alternative	“a”, “b”, “c”
[-]	[a-c]	Opseg	“a”, “b”, “c”
[^]	[^abc]	Alternative koje ne odgovaraju navedenom	“d”, “e”,….(ostalo iz azbuke)
{m, n}	a{1, 3}	Broj ponavljanja	“a”, “aa”, “aaa”
.	a.	Bilo koji znak (osim nove linije) *[a.b] – tu se često interpetira baš kao znak “.”*	“aa”, “ab”, “ac”…
\	a\.	Escape, specijalne karaktere tretiramo kao obične	“a.”

Ideja

MI: Leksika se predstavi regularnim izrazima

TEORIJA: Za svaki regularni izraz postoji odgovarajući konačni automat i zna se precizan postupak kako se on formira.

ALAT: Generiše funkciju koja simulira rad odgovarajućeg automata koji će biti zadužen za prepoznavanje

$\longrightarrow$
napišemo regularne izraze
dobijemo gotovu mašinu / alat / implementaciju

Postupak

1. Regularni izrazi koji predstavljaju gramatiku leksike se navode u specifikaciji (`jezik.l`)

2. Specifikacija se prosleđuje flex programu. [`$ flex jezik.l`]

3. Rezultat je izlazni fajl (lex.yy.c) u kom je definisana funkcija `int yylex(void)`

4. Nakon kompajliranja dobija se izvršni fajl [`$ gcc -o skener lex.yy.c`]

5. Pokreće se

- interaktivno – ulaz je tastatura (`$ ./skener`)
- vrši se analiza ulaznog fajla (`$ ./skener < test.txt`)

Primer

L = {0, 1, 42, 402, 000150...}

Jezik L - skup svih reči nad azbukom koju čine cifre, bez prazne reči

Regularni izrazi:

[0-9]+

jezik.l

```
%option noyywrap

%%

[0-9]+

%%

int main() {
   yylex();
}
```

Opcije i definicije

Pravila

Korisnički kod

Šta ćemo ovim dobiti?

```
%option noyywrap

$ flex –-noyywrap jezik.l
```

Kada flex dobije indikaciju da je završeno čitanje sa jednog ulaza tada on proverava vrednost funkcije yywrap().

Povratna vrednost ukazuje na to da li se nastavlja sa sledećim ulazom (npr. novi fajl) [povratna vrednost je 0] ili je obrada završena [povratna vrednost je 1].

Ako nije definisana ova funkcija onda:

- ili je potrebno definisati gore navedenu opciju (kao da je yywrap vratio 1)
- ili se mora linkovati biblioteka koja ima ovu funkciju (libfl bibiloteka, povratna vrednost 1)

Pokretanje flex-a:

```
$ flex jezik.l
```

Nakon izvršavanja kreiran je fajl `yy.lex.c` (> 1500 redova koda)

```
#line 3 "lex.yy.c"

#define  YY_INT_ALIGNED short int

/* A lexical scanner generated by flex */

#define FLEX_SCANNER
#define YY_FLEX_MAJOR_VERSION 2
#define YY_FLEX_MINOR_VERSION 6
#define YY_FLEX_SUBMINOR_VERSION 4
#if YY_FLEX_SUBMINOR_VERSION > 0
#define FLEX_BETA
#endif

/* First, we deal with  platform-specific or compiler-specific issues. */

/* begin standard C headers. */
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <stdlib.h>

/* end standard C headers. */

/* flex integer type definitions */
```

L = {0, 1, 42, 402, 000150...}

Jezik L - skup svih reči nad azbukom koju čine cifre, bez prazne reči

Da li se isplati?

```
#line 3 "lex.yy.c"

#define  YY_INT_ALIGNED short int

/* A lexical scanner generated by flex */

#define FLEX_SCANNER
#define YY_FLEX_MAJOR_VERSION 2
#define YY_FLEX_MINOR_VERSION 6
#define YY_FLEX_SUBMINOR_VERSION 4
#if YY_FLEX_SUBMINOR_VERSION > 0
#define FLEX_BETA
#endif

/* First, we deal with  platform-specific or compiler-specific issues. */

/* begin standard C headers. */
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <stdlib.h>

/* end standard C headers. */

/* flex integer type definitions */
```

Kompajliranje funkcije:

```
$ gcc lex.yy.c
$ gcc -o jezik lex.yy.c
```

Opcija –o : definisanje naziva programa. Ako je izostavljeno biće kreiran a.out fajl.

Pokretanje skenera:

```
$ ./jezik
```

ctrl+D - izlazak

- sve dok ulazni string odgovara nekom regularnom izrazu:
  ne postoji podrazumevana akcija,
  odnosno pročitani tekst se preskače i nastavlja se sa radom
  (za sada je sve u redu, nastavi)

- ukoliko pročitano ne odgovara nijednom regularnom izrazu:
  podrazumevana akcija je kopiranje tog teksta na izlaz

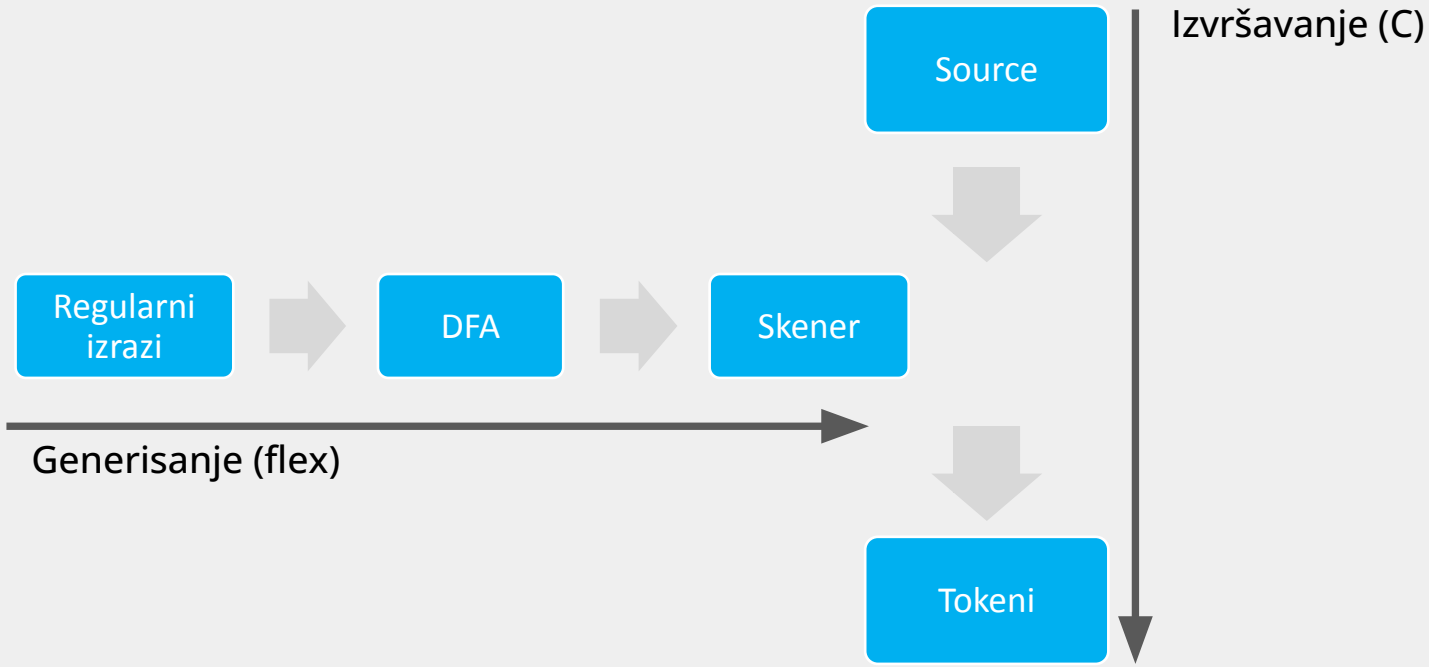
```
123

abc
abc
123abc
abc
ab123c
abc
```

jezik.l

```
%option noyywrap

%%

[0-9]+    { printf("Broj!"); }

%%

int main() {
   yylex();
}
```

```
123
Broj!
abc
abc
123abc
Broj!abc
ab123c
abBroj!c
```

Možemo dodati korisničke akcije


Source
Izvršavanje (C)
Regularni izrazi
DFA
Skener
Generisanje (flex)
Tokeni

Rešenja za neke probleme:

- “<=“ treba prepoznati kao jedan operator <=, a ne kao dva: < i =

  “Maximal munch” rule
  Pronalazi se najduži string koji odgovara nekom RE

- “if” treba prepoznati kao ključnu reč, a ne kao identifikator koji je definisan, na primer, kao: [a-zA-Z][a-zA-Z0-9]*

  Bira se prvi pronađeni RE → patern za “if” treba navesti pre paterna za identifikator

- Flex nije jedini alat
- Nastao kao unapređenje Lex alata
- Koriste se i drugi: Antlr, re2c, javaCC,...

Prednosti i mane korišćenja alata za generisanje skenera naspram samostalno (ručno) implementiranog skenera?

Zadaci (neobavezno)

Pogledati Antlr.

Kako izgledaju formalne specifikacije leksike nekih popularnih jezika?

Linkovi Flex:
https://github.com/westes/flex/
https://westes.github.io/flex/manual/

Pogledati zbirke.